

Technical Documentation

Automated Deployment Pipeline: doms-recon

1. Architecture Overview

This environment utilizes Infrastructure as Code (IaC) to automatically provision a secure, two-tier network architecture. It separates public-facing traffic routing from backend application processing, mimicking an enterprise environment where backend systems are managed remotely and securely.

- **Host Machine:** Developer Workstation
- **Hypervisor:** VirtualBox (Managed via Vagrant)
- **Node 1 (Gateway):** Ubuntu 22.04 LTS | IP: `192.168.50.10` | Service: Nginx Reverse Proxy
- **Node 2 (App Server):** Ubuntu 22.04 LTS | IP: `192.168.50.11` | Service: Docker & Streamlit

2. The Application Payload

The core workload is a custom Python application (`doms-recon`), containerized via Docker to ensure consistent execution regardless of the underlying host operating system.

Streamlit is utilized to serve the Python backend logic as an interactive, real-time web dashboard exposed on port `8501`.

3. Infrastructure & Provisioning

The environment is entirely automated using Vagrant and Bash scripting, eliminating the need for manual server configuration.

- **Vagrantfile:** Defines the hardware specifications (RAM/CPU), assigns static IPs on a private internal network, and maps port `8080` on the host to port `80` on the Gateway.
- **setup-app.sh:** A Bash script executed automatically on the App Server upon boot. It updates the OS, installs Docker and Git, pulls the latest application code from GitHub, and builds/runs the Docker container in detached mode with auto-restart policies.
- **setup-gateway.sh:** A Bash script executed on the Gateway. It installs Nginx and configures a reverse proxy block to securely bridge external HTTP traffic and WebSockets from port `80` to the hidden App Server's port `8501`.

4. Troubleshooting Ledger

During the initial build phase, several environmental and routing anomalies were encountered and resolved. This ledger documents the root causes and applied solutions.

Error / Symptom	Root Cause	Resolution
Missing Directory Error during Docker Build	The <code>git clone</code> command downloaded the repository into a default folder instead of the specific <code>/opt/</code> path expected by the subsequent Docker build command.	Appended the explicit target directory path (<code>/opt/doms-recon</code>) to the end of the <code>git clone</code> command in the Bash script.
Browser ERR_CONNECTION_TIMED_OUT	The host machine's network adapter failed to route browser traffic directly to the VirtualBox Host-Only private IP subnet (<code>192.168.50.11</code>).	Implemented a Gateway VM using Vagrant port forwarding (<code>localhost:8080 → guest:80</code>) to bypass host routing limitations.
502 Bad Gateway (Nginx)	VirtualBox failed to assign the static IP (<code>192.168.50.11</code>) to the App Server's network interface during the heavy initial boot/provisioning cycle.	Executed <code>vagrant reload appserver</code> to force hardware re-initialization, then manually ran <code>docker start doms-container</code> to wake the application.

Error / Symptom	Root Cause	Resolution
Blank Gray Screen & WebSocket Failure	Streamlit's built-in CORS and Cross-Site Request Forgery (XSRF) security protocols actively blocked the connection because the traffic originated from a proxy instead of its native IP.	Appended -- <code>server.enableCORS=false</code> and -- <code>server.enableXsrfProtection=false</code> flags to the <code>docker run</code> command to authorize the Nginx proxy traffic.
Persistent WebSocket Disconnect	The Nginx configuration included a trailing slash in the proxy pass URL, which stripped the routing structure required by Streamlit's <code>/_stcore/stream</code> endpoint.	Removed the trailing slash and implemented a dedicated location <code>^~ /_stcore/stream</code> block in Nginx with explicit Upgrade headers for WebSocket traffic.